

FIELD REFERENCE · v1.0 · 2026

RAG & Agentic AI Systems Cheat Sheet

A single-reference guide for designing production RAG pipelines and agentic systems — retrieval, chunking, tool calling, ReAct, memory, guardrails, and LoRA fine-tuning — aligned to current industry practice.

RAG Pipeline

Chunking

Hybrid Retrieval

Reranking

Tool Calling

ReAct Agents

Memory

Guardrails

LoRA / PEFT

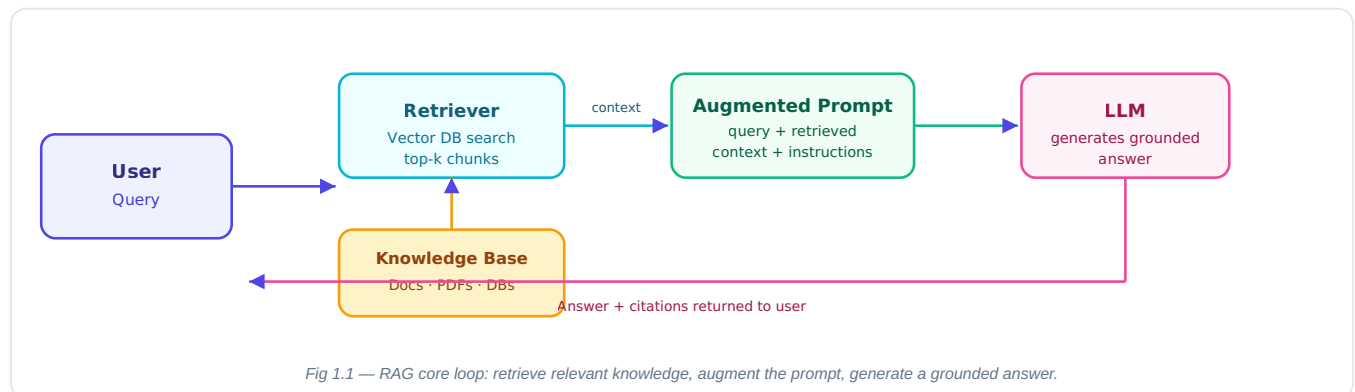
Evaluation

Table of Contents

01	What is RAG? Core Concept	3
02	End-to-End RAG Architecture	4
03	Chunking Strategies	5
04	Embeddings & Vector Databases	6
05	Retrieval Strategies (Dense / Sparse / Hybrid)	7
06	Reranking & Advanced RAG Patterns	8
07	RAG Evaluation Metrics	9
08	Tools & Function/Tool Calling	10
09	ReAct Architecture (Reason + Act)	11
10	Agent Memory Techniques	12
11	Guardrails — Implementation Guide	13
12	LoRA / PEFT Fine-Tuning	14
13	Putting It Together — Full Agentic RAG Stack	15
14	Quick Reference & Tech Stack Cheat Table	16

What is RAG?

Retrieval-Augmented Generation (RAG) grounds an LLM's answers in external, up-to-date, or private knowledge by retrieving relevant documents at query time and inserting them into the prompt — instead of relying only on the model's frozen training data.



Why RAG instead of just a bigger model?

Freshness

Injects current / private data without retraining. Knowledge base updates instantly reflect in answers.

Grounding

Reduces hallucination by citing real source chunks; answers can be traced back to documents.

Cost

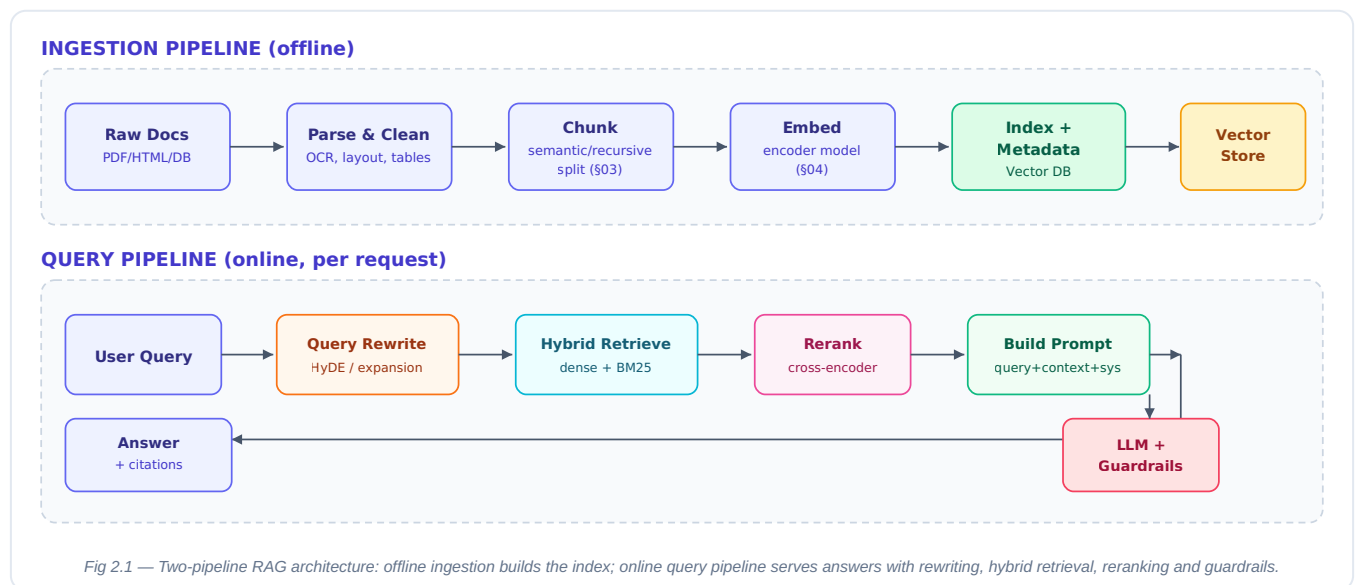
Cheaper than fine-tuning for knowledge injection — no GPU training runs needed to "teach" new facts.

RAG vs Fine-Tuning vs Long-Context — When to use what

Need	Best approach	Why
Answer from private/changing docs	RAG	Swap knowledge base without retraining; traceable sources
Teach a new skill, tone, or output format	LoRA / Fine-tuning	Changes model <i>behavior</i> , not just knowledge
Small, fixed doc set every query needs fully	Long-context prompting	Simpler pipeline if it fits the context window and cost budget
Enterprise-scale, evolving corpus + behavior control	RAG + LoRA + Guardrails	Combine all three — this is the modern standard stack (see §13)

End-to-End RAG Architecture

Every production RAG system has two independent pipelines: an **offline Ingestion Pipeline** that builds the index, and an **online Query Pipeline** that serves user requests.



Design Checklist

- DO** Version your index (embedding model + chunk config) so you can re-embed on model upgrades.
- DO** Store rich metadata (source, section, date, permissions) alongside vectors for filtering.
- DO** Log retrieved chunks per query for offline evaluation and debugging.
- DON'T** Retrieve with only cosine similarity on raw chunks — always rerank for production quality.
- DON'T** Skip access-control filtering at retrieval time in multi-tenant systems.
- DON'T** Hard-code top-k; tune it per use case (typically k=3–10 after reranking).

Chunking Strategies

Chunk size and split method directly determine retrieval quality. The goal: each chunk should be a self-contained, semantically coherent unit small enough to retrieve precisely, large enough to preserve context.

Strategy	How it works	Best for	Typical size
Fixed-size	Split every N tokens/chars with overlap	Quick prototypes, uniform text	256–512 tokens, 10–20% overlap
Recursive character split	Try paragraph → sentence → word boundaries in order	General-purpose default (LangChain/ LlamaIndex default)	500–1000 chars
Semantic chunking	Split where embedding similarity between adjacent sentences drops	Long-form docs, articles, mixed topics	Variable
Document-structure aware	Split on markdown headers / HTML tags / slide boundaries	Technical docs, wikis, manuals	Per section
Sentence-window	Embed single sentences; retrieve with N neighboring sentences attached	High-precision QA	1 sentence + window
Parent-document	Embed small child chunks; return the larger parent chunk/ doc for context	Balancing retrieval precision with generation context	Child 200–400, parent full section
Table/code-aware	Keep tables, code blocks, and rows atomic — never split mid-structure	Technical & financial documents	Whole unit

Rule of thumb: start with recursive splitting at ~512 tokens / 10-15% overlap, measure retrieval quality (\$07), then move to semantic or parent-document chunking if precision or context loss is a problem.

Overlap Concept

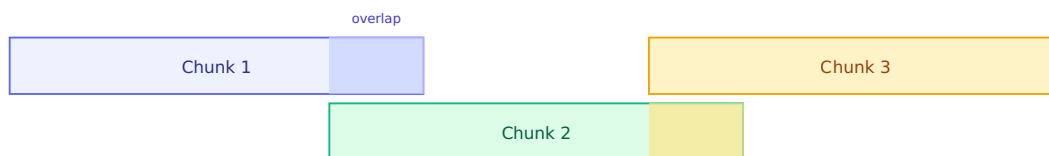


Fig 3.1 — Overlapping windows preserve context that would otherwise be cut at a chunk boundary.

Embeddings & Vector Databases

Choosing an Embedding Model (2026 landscape)

Model family	Notes
OpenAI <code>text-embedding-3</code>	Strong general baseline, hosted API, variable dimensions (Matryoshka)
Cohere <code>embed-v4</code>	Multilingual, strong retrieval benchmark performance, hosted
Voyage AI embeddings	Domain-tuned variants (code, finance, legal), hosted
BGE / GTE / E5 (open-source)	Self-hostable, top of MTEB leaderboard, good for data-sovereignty needs
Matryoshka embeddings	Truncatable vectors — trade dimension size for speed/storage at query time

Vector Database Options

Managed / Cloud

Pinecone, Weaviate Cloud, Qdrant Cloud, Zilliz (Milvus) — fast to start, handles scaling & sharding.

Add-on to existing DB

pgvector (Postgres), Redis, Elasticsearch/OpenSearch (also gives BM25 for hybrid search), MongoDB Atlas Vector Search.

Self-hosted OSS

Milvus, Qdrant, Weaviate, Chroma — full control, needed for on-prem/ data-residency requirements.

Index algorithms

HNSW (default, best recall/speed), IVF-Flat, IVF-PQ (compression at scale) — most DBs pick this for you.

Metadata Filtering

Always combine vector similarity with structured metadata filters (`tenant_id`, date range, doc type, access level) — this is standard in production and prevents cross-tenant leakage and stale-document retrieval.

```
results = vector_db.query(
    vector=query_embedding,
    top_k=20,
    filter={"tenant_id": "acme", "doc_type": "policy", "status": "active"}
)
```

Retrieval Strategies: Dense · Sparse · Hybrid

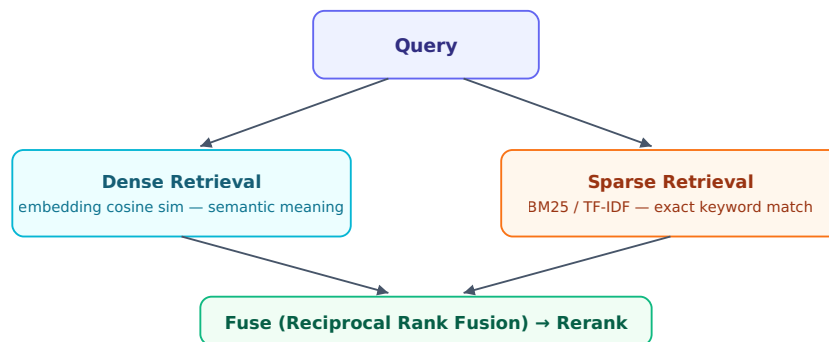


Fig 5.1 — Hybrid search combines semantic recall (dense) with exact-match precision (sparse/keyword), fused via RRF.

Method	Strength	Weakness
Dense (embeddings)	Captures meaning/synonyms, cross-lingual	Misses exact IDs, codes, rare terms, numbers
Sparse (BM25/keyword)	Exact term match, acronyms, part numbers	No semantic understanding, vocabulary mismatch
Hybrid (both + RRF)	Best of both — current industry default	Slightly more infra (needs keyword index too)

Query Transformation Techniques

HyDE (Hypothetical Doc Embeddings)

LLM writes a hypothetical answer first, then embeds *that* to search — closes vocabulary gap between short queries and long documents.

Query decomposition

Break a complex multi-part question into sub-questions, retrieve separately, then synthesize.

Multi-query

LLM generates 3-5 paraphrased versions of the query, retrieves for each, merges results — improves recall.

Step-back prompting

Ask a more general question first to retrieve broader context, then answer the specific one.

Reranking & Advanced RAG Patterns

Reranking

Initial retrieval (dense/sparse) optimizes for *speed* across millions of vectors. A **cross-encoder reranker** then re-scores the top ~20-50 candidates by jointly encoding (query, chunk) pairs for much higher precision before the final top-k reach the LLM.

Reranker	Type
Cohere Rerank	Hosted API, strong general performance
BGE-reranker / Jina Reranker	Open-source, self-hostable cross-encoders
Cross-encoder (MiniLM etc.)	Lightweight, fast local reranking
LLM-as-reranker	Prompt an LLM to score/order chunks — flexible but slower/costlier

Advanced RAG Patterns

Pattern	What it adds
Self-RAG	Model reflects on whether retrieval is needed and critiques its own generated answer against retrieved evidence
Corrective RAG (CRAG)	Grades retrieved docs for relevance; if weak, triggers a web search or query rewrite before generating
Agentic RAG	An agent decides <i>which</i> tool/retriever to call, can loop retrieval multiple times (see §09 ReAct)
GraphRAG	Builds a knowledge graph over entities/relations for multi-hop reasoning across documents
Contextual retrieval	Prepends LLM-generated chunk-level context (doc summary) before embedding, boosting retrieval accuracy

Modern default stack: Recursive/semantic chunking → embed with a top-MTEB model → hybrid (dense+BM25) retrieval → cross-encoder rerank → top 3-8 chunks into prompt → guardrails → LLM.

RAG Evaluation Metrics

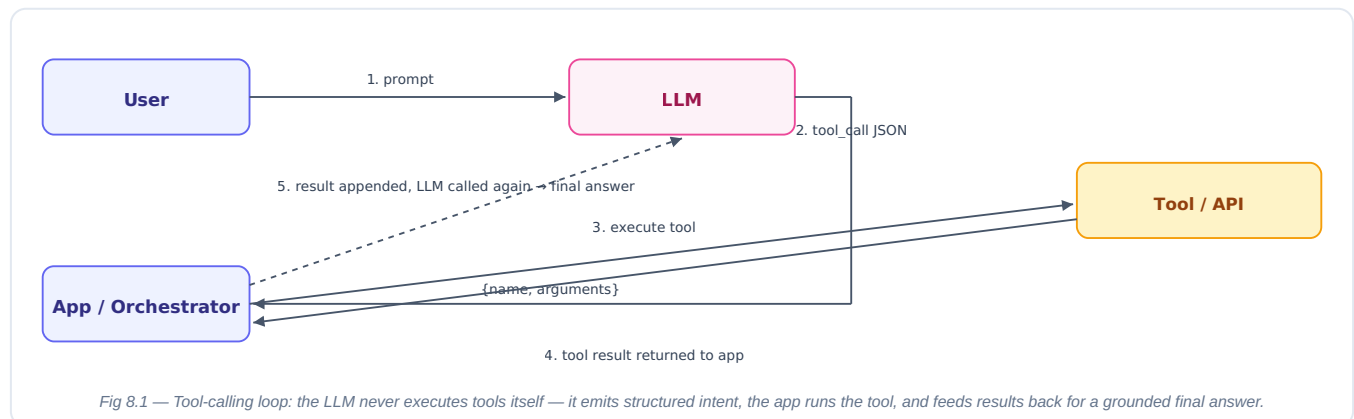
Use a framework like **RAGAS**, **TruLens**, or **DeepEval** to score both the retriever and the generator automatically, ideally with an LLM-as-judge plus a labeled test set.

Metric	Measures	Component
Context Precision	Are retrieved chunks actually relevant to the query?	Retriever
Context Recall	Did retrieval find all the info needed to answer?	Retriever
Faithfulness / Groundedness	Is the generated answer fully supported by retrieved context (no hallucination)?	Generator
Answer Relevancy	Does the answer actually address the question asked?	Generator
Answer Correctness	Does it match ground truth (needs labeled data)?	End-to-end
Latency / Cost per query	Retrieval + rerank + generation time and token spend	System

Track these per release in a regression test set — RAG quality silently degrades when chunking, embedding model, or prompt templates change.

Tools & Tool / Function Calling

Tool calling lets an LLM invoke external functions (APIs, calculators, search, RAG retrievers, DB queries) mid-conversation. The model outputs a structured JSON call; your application executes it and feeds the result back.



Anatomy of a Tool Definition (JSON Schema)

```

{
  "name": "search_orders",
  "description": "Look up a customer's order history by email.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "limit": {"type": "integer", "default": 5}
    },
    "required": ["email"]
  }
}

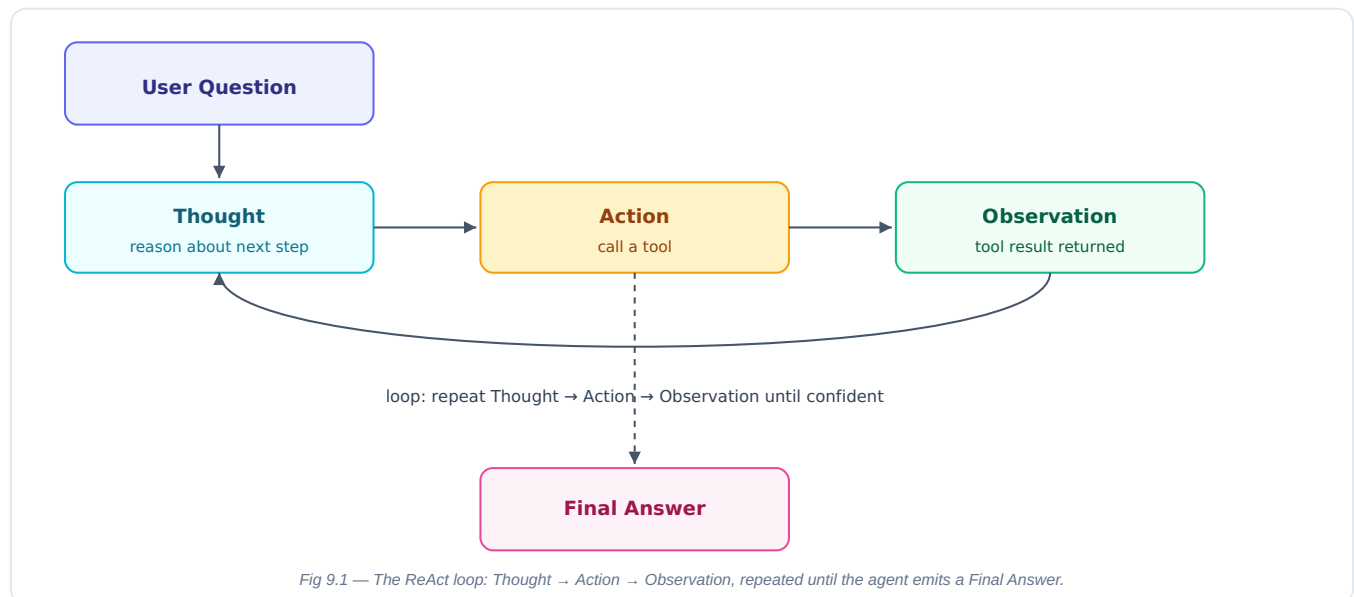
```

Tool Design Best Practices

- DO** Write tool *descriptions* as clearly as you'd write docs for a new teammate — the model reasons from these alone.
- DO** Keep tools narrow and single-purpose; compose behavior, don't build mega-tools.
- DO** Return structured, compact results (not full HTML pages) to save context tokens.
- DON'T** Expose destructive tools (delete, send, pay) without a human-confirmation step or guardrail.
- DON'T** Give the model more than ~15-20 tools at once — retrieval-augmented tool selection helps at scale.
- DON'T** Trust tool arguments blindly — validate/sanitize before execution (see §11 Guardrails).

ReAct Architecture (Reason + Act)

ReAct interleaves chain-of-thought *reasoning* with tool *actions* in a loop: the model thinks, acts, observes the result, and repeats until it has enough information to answer — this is the foundation of most modern agent frameworks.



Prompt Skeleton

```

# System instructions define the loop format
You have access to tools: [search, calculator, get_weather]
Use this format strictly:

Thought: reason about what to do next
Action: tool_name
Action Input: {arguments as JSON}
Observation: {result - filled in by the system}
... (repeat Thought/Action/Observation as needed)
Thought: I now know the final answer
Final Answer: {answer to the user}
  
```

Implementation Notes

Concern	Practice
Loop control	Set a <code>max_iterations</code> cap (typically 5-10) to prevent infinite loops and runaway cost
Native tool calling	Modern LLM APIs (Claude, GPT, Gemini) support structured tool-call output directly — no need to hand-parse "Action:" text
Frameworks	LangGraph, LlamaIndex Agents, CrewAI, or a hand-rolled loop with the provider's native tool-use API
Error handling	Feed tool errors back as Observations so the agent can retry or choose a different action
Parallel actions	Advanced agents can dispatch multiple independent tool calls in one step and merge observations

Agent Memory Techniques

LLMs are stateless — "memory" is engineered by the application: what to keep in the context window, and what to store/retrieve externally.



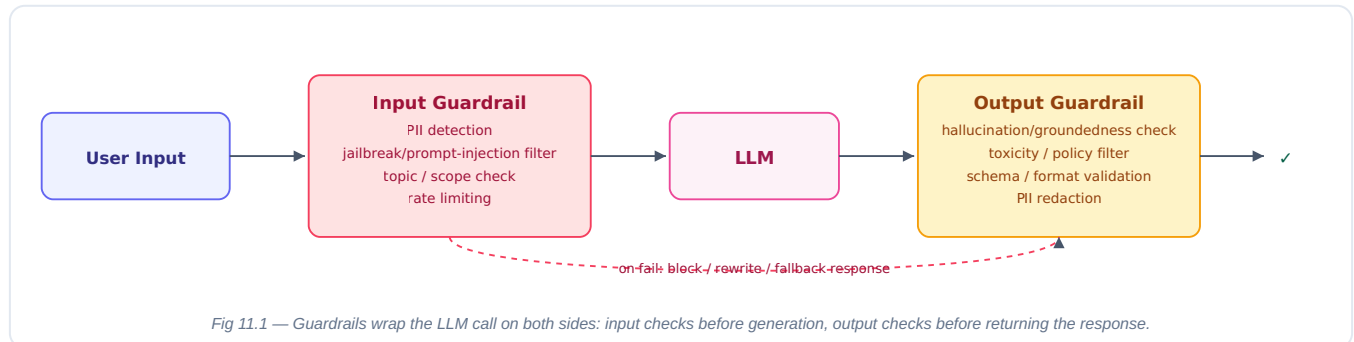
Fig 10.1 — Four memory types feeding an agent turn: short-term context, episodic history, semantic facts, and procedural skills.

Technique	Description	When to use
Full history in context	Pass entire conversation every turn	Short sessions, fits context window
Sliding window	Keep only the last N turns	Simple chat, bounded cost
Summarization memory	Periodically compress older turns into a running summary	Long sessions, cost control
Vector-store memory (RAG-as-memory)	Embed past interactions/facts; retrieve relevant ones each turn	Long-term personalization, multi-session recall
Structured/key-value memory	Extract facts into a schema (user prefs, entities) instead of raw text	Precise, queryable long-term profile
Scratchpad / working memory	Explicit space for the agent's intermediate reasoning & tool outputs within a task	Multi-step agent tasks (pairs with ReAct)

Production pattern: sliding window + auto-summarization for short-term, vector-store memory for long-term facts, retrieved via a lightweight RAG query at the start of each turn.

Guardrails — Implementation Guide

Guardrails are checks applied **before** the LLM sees input and **after** it produces output, to enforce safety, correctness, format, and business rules.



Guardrail Layers to Implement

Layer	Checks	Common tools
Input validation	PII/secret detection, prompt-injection & jailbreak patterns, off-topic/scope filtering, length & rate limits	Guardrails AI, NeMo Guardrails, Lakera, Presidio (PII)
Retrieval-layer	Access-control filters, source allow-listing, freshness checks	Vector DB metadata filters
Output validation	Groundedness/faithfulness vs. retrieved context, toxicity/bias, JSON schema conformance, competitor/banned-topic filters	Guardrails AI, NeMo Guardrails, custom LLM-judge
Tool-execution	Argument validation, permission checks, human-in-loop for destructive actions, sandboxing	Custom policy layer around tool dispatch
System-level	Fallback responses, circuit breakers, audit logging, red-team test suite in CI	Observability stack (LangSmith, Arize, etc.)

Minimal Implementation Pattern

```
def answer(query, user_ctx):
    # 1. Input guardrail
    if contains_injection(query) or out_of_scope(query, user_ctx):
        return safe_fallback_message()

    context = retrieve_and_rerank(query, user_ctx.permissions)
    draft = llm.generate(query, context)

    # 2. Output guardrail
    if not is_grounded(draft, context) or is_unsafe(draft):
        draft = llm.regenerate_stricter(query, context)
    return redact_pii(draft)
```

Layer guardrails: cheap regex/classifier checks first (fast, low-cost), LLM-based checks second (slower, higher-recall) only when needed.

LoRA / PEFT Fine-Tuning

LoRA (Low-Rank Adaptation) freezes the base model's pretrained weights and injects small trainable low-rank matrices into each layer, so fine-tuning only updates a tiny fraction of parameters — cheap, fast, and easy to swap.

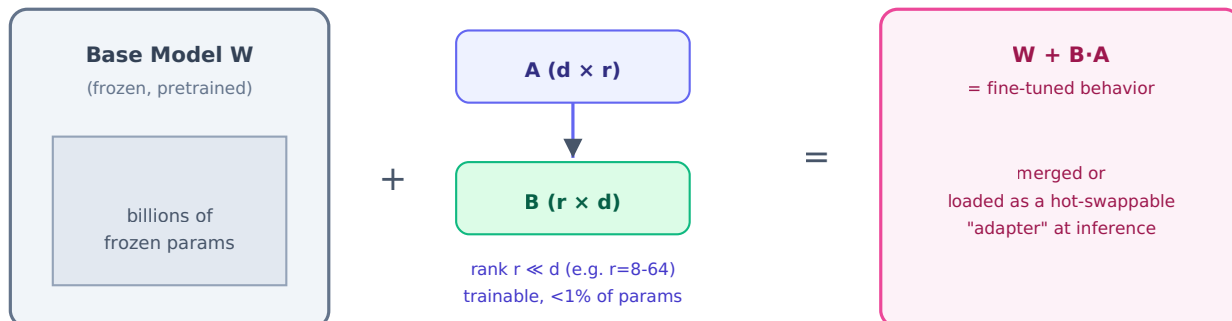


Fig 12.1 — LoRA adds two small low-rank matrices (A, B) alongside frozen weights W; only A and B are trained.

Key Hyperparameters

Parameter	Typical range	Effect
rank (r)	4 – 64	Higher = more capacity & params, risk of overfitting on small data
alpha	usually 2× rank	Scaling factor for the adapter's contribution
target_modules	attention q_proj/v_proj (+ MLP for more capacity)	Which weight matrices get adapters
dropout	0.0 – 0.1	Regularization for small datasets
QLoRA	4-bit base + LoRA	Fine-tune large models on a single consumer/prosumer GPU

LoRA vs RAG — Combine, Don't Choose

Use LoRA for

Tone/persona, output format/schema adherence, domain-specific reasoning style, task specialization (e.g. classification), efficient multi-tenant adapters (swap per customer).

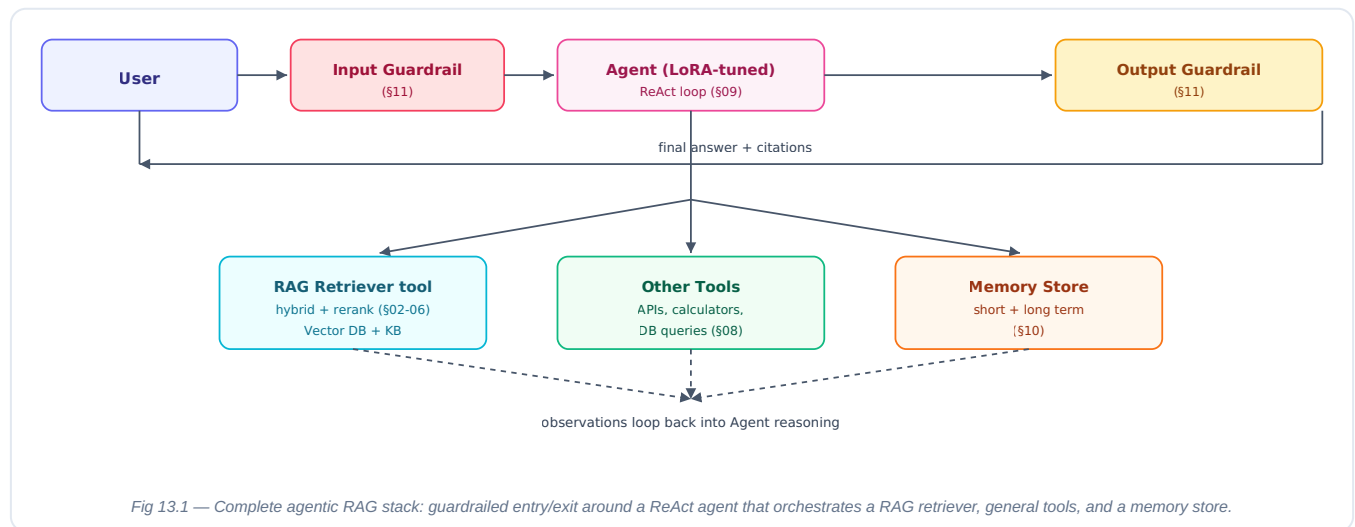
Use RAG for

Injecting facts, keeping knowledge current, source-grounded/citable answers, large or frequently changing corpora.

Modern best practice: **LoRA-tune the model's behavior** (how it reasons, formats, uses tools) + **RAG for knowledge** (what it knows) — they're complementary, not competing techniques.

Putting It Together — Full Agentic RAG Stack

A modern production system layers all the pieces above into one loop: guardrailed input → agent reasoning (ReAct) → tool calls including a RAG retriever → memory read/write → guardrailed output.



Build Order (recommended)

1. Ship a plain RAG pipeline first (§02-07) and get retrieval quality solid.
2. Add tool calling for actions beyond retrieval (§08).
3. Wrap in a ReAct loop once multi-step reasoning is needed (§09).
4. Add memory once multi-turn/multi-session context matters (§10).
5. Add guardrails from day one on input/output, harden as you learn failure modes (§11).
6. Only invest in LoRA once you have a clear behavior gap that prompting/RAG can't fix (§12).

Quick Reference & Tech Stack Cheat Table

Layer	Popular tools / libraries
Orchestration	LangChain, LlamaIndex, LangGraph, Haystack, CrewAI, custom loop w/ native tool APIs
Embeddings	OpenAI, Cohere, Voyage AI, BGE/E5/GTE (open-source)
Vector DB	Pinecone, Qdrant, Weaviate, Milvus, Chroma, pgvector
Keyword/hybrid	Elasticsearch, OpenSearch, BM25 (rank_bm25)
Reranking	Cohere Rerank, BGE-reranker, Jina Reranker
Fine-tuning (PEFT)	HuggingFace PEFT, Unsloth, Axolotl
Guardrails	Guardrails AI, NeMo Guardrails, Presidio, Lakera
Evaluation	RAGAS, TruLens, DeepEval, Arize Phoenix
Observability	LangSmith, Arize, Langfuse, Helicone

End-to-End Checklist Before Shipping

- ☐ Chunking strategy tested against real queries
- ☐ Hybrid retrieval + reranker in place
- ☐ Access-control metadata filters on retrieval
- ☐ Tool descriptions reviewed, destructive tools gated
- ☐ Input & output guardrails wired in
- ☐ Memory strategy fits session length & cost budget
- ☐ RAGAS/eval suite running in CI on every prompt/index change
- ☐ Logging & observability for every retrieval + tool call

Keep this cheat sheet updated — this is a fast-moving field. Revisit embedding models, reranker choices, and framework versions every few months.